CT4100 Information Retrieval Assignment 2

Gavin Skehan – 21440824 | Evan Murphy - 21306306

Question 1

Given a query that a user submits to an IR system and the top N documents that are returned as relevant by the system, devise an approach (high-level algorithmic steps will suffice) to suggest query terms to add to the query. Typically, we wish to give a large range of suggestions to the users capturing potential intended query needs, i.e., high diversity of terms that may capture the intended query context/content.

Approach:

1. Parse the user's query:
    a. The initial step is to ensure there are no syntax errors, making it easy for the program to correctly match the query to the chunks in the document. Syntax errors may lead to important query terms not being recommended.
    b. Normalize the user's query by lowercasing, removing stop words, and applying stemming or lemmatization. Tokenize the query into meaningful components for later analysis.

2. Retrieve top N relevant documents:
    a. Using an inverted index for efficient document retrieval, we can identify the documents that we can target for the processing stage, this can help reduce computational costs as we can remove any documents that may carry no value for adding query terms.
    b. To rank the documents in terms of perceived relevancy we can use BM25 “to estimate the relevance of documents to a given search query” [1]. We can then return the top n documents or define a threshold so the documents over x are returned in order. This will also help the computational costs of the algorithm.

3. Process the retrieved documents:
    a. TF-IDF is the term frequency-inverse document frequency, it measures the importance or relevance of a term.
    b. Identify co-occurring terms (terms that appear frequently together).
    c. Represent the position of the document and the query in a vector space model to compute the similarity. Rank the documents based on the cosine similarity score to determine the nearness to the query. The closer the query is to a document the greater the similarity score.

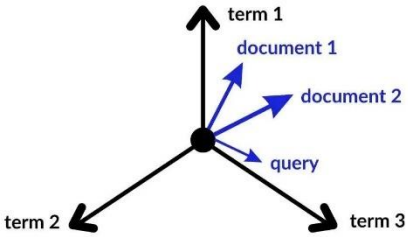

Figure 1: Vector Space Model Example [3]: similarity determined by distance from query to document x and y.

4. Analyse historical queries:
    a. Identify terms that overlap and extract the most popular/trending terms to see what users are commonly searching by analysing historical query logs.

5. Semantic Analysis and Clustering:
    a. Generate semantic embeddings for all extracted terms using Word2Vec to capture contextual similarities.
    b. Perform clustering (k-means) to group semantically similar terms to ensure diverse query expansion suggestions.

6. Select representative terms:
    a. Select representative terms from clusters based on their centrality within the embedding space, which indicates their importance. Terms located near the cluster centroid are considered more representative of the cluster's overall theme. These terms are potential query suggestions, prioritized using TF-IDF scores.
7. Rank selected terms:
    a. Rank selected terms based on relevance to the initial query (cosine similarity), popularity in historical logs and frequency in retrieved documents.

8. Present suggestions to the user:
    a. Display a diverse list of suggested terms to the user, covering a broad range of topics that align with the query. This ensures maximum diversity in suggestions and provides the user with a heuristic understanding of potential query expansions.

9. Iterate
    a. Allow the user to make adjustments to the current query after gathering suggestions from the algorithm. This is an optional step, if the user decides to alter the query the values of the TF-IDF and similarity scores will have to be updated too.

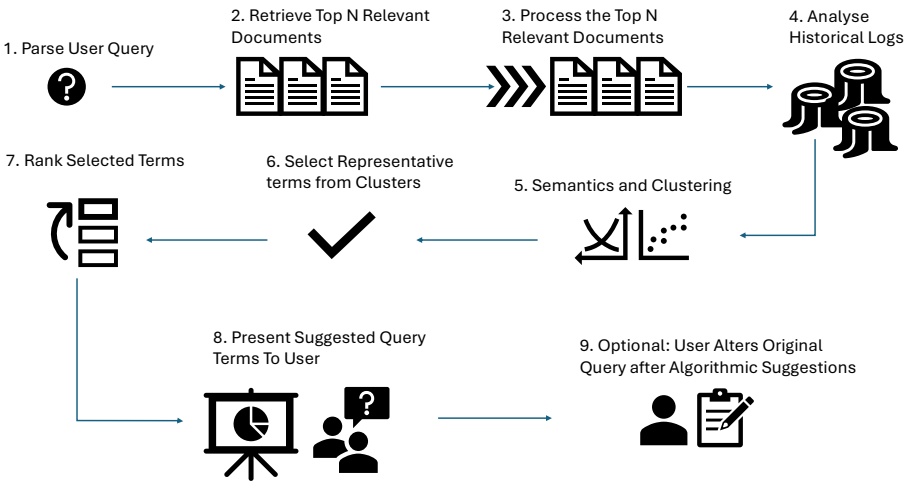

Figure 2: Flow Chart of Algorithm

Question 2

Consider the following scenario: a company search engine is employed to allow people to search a large repository. All queries submitted to the system are recorded. A record that contains the id of the user and the term in the query is stored. The order of the terms is not stored, and neither is any timestamp. Each entry in this record is effectively an id and a set of terms.

The designers of the search engine, decide to use this information to develop an approach to make query term suggestions for users, i.e., at run time, once a user has entered their query terms, the system will suggest potential extra terms to add to the query.

Given the data available, outline an approach that could be adopted to generate these suggested terms. A brief outline is sufficient to capture the main ideas in your approach. The designers of the system wish to take into account previous queries and any similarities between users. Identify the advantages and disadvantages of your approach (briefly).

Outline of Approach

Considering the data that is available to us, which is a record containing the user ID and terms in a query, for each query, we have outlined below a simple approach for generating user queries, considering the previous queries and any similarities between users.

1) Construct a user-term matrix
    a. This would be a matrix with rows representing users and columns representing query terms.
    b. Each cell of the matrix indicates if a user has used a specific term in any of their queries; this is indicated using ‘0’ if the term was not present, and ‘1’ if it has been used

c. This creates a sparse matrix that can be used in the next steps.

2) Calculate user similarity
    a. We would compute a similarity score between each pair of users based on query terms that they share.
    b. A good approach for doing this could be Cosine Similarity. This involves calculating the cosine of the angle between the two user vectors in the user-term matrix.
    c. Cosine similarity will ensure that users who have overlapping term vectors at different scales are still recognized as being similar. One user might search for a broader range of topics leading to a longer vector, but this does not mean that they are dissimilar from a user with a smaller vector. Cosine similarity handles this by normalizing the vectors, disregarding the magnitude and instead focusing on the angle between them.[4]

3) Identify user neighbourhoods
    a. This would involve identifying a neighborhood of similar users for each user, based on the similarity scores calculated for them.
    b. An effective way to do this would be correlation thresholding or getting the Best-N correlations to find the most similar users to the target user. These approaches should both be tested to determine which creates the best neighborhoods.
    c. I believe the Best-N approach would be preferred, as performance can be balanced and computational efficiency improved.
    d. Clustering could be informed on each neighbourhood based on the term usage of different users. When suggesting a term in the next step, there are multiple clusters in the neighbourhood to draw recommendations from which can make the system more *diverse*.

4) Generate term suggestions
    a. Analysing the query terms by the users in the target neighbourhood, we will find terms frequently used by neighbours but not yet used by the target user. These terms become potential suggestions for the target users' query.
    b. By analyzing terms that are used frequently in a user's neighbourhood, the model can identify patterns and terms that are most likely to enhance the users search results, boosting their query experience.
    c. As previous queries by the target user might be relevant to them again, when scoring queries and assigning a weight to them we would apply a

higher weight to historical search terms by the user, as they may have a similar query again.

5) Rank the suggestions
    a. We would order the suggested terms based on a score that is computed based on a variety of factors, including: frequency of the use within the neigbourhood, relevance to the target user's current query, and diversity relative to the targets users existing query terms.
    b. The ranking formula should consider how relevant a term is but could also incorporate a function whereby each term has a diversity or novelty score which is geared towards helping the user explore new topics they would not have considered relevant but might be useful to their search.

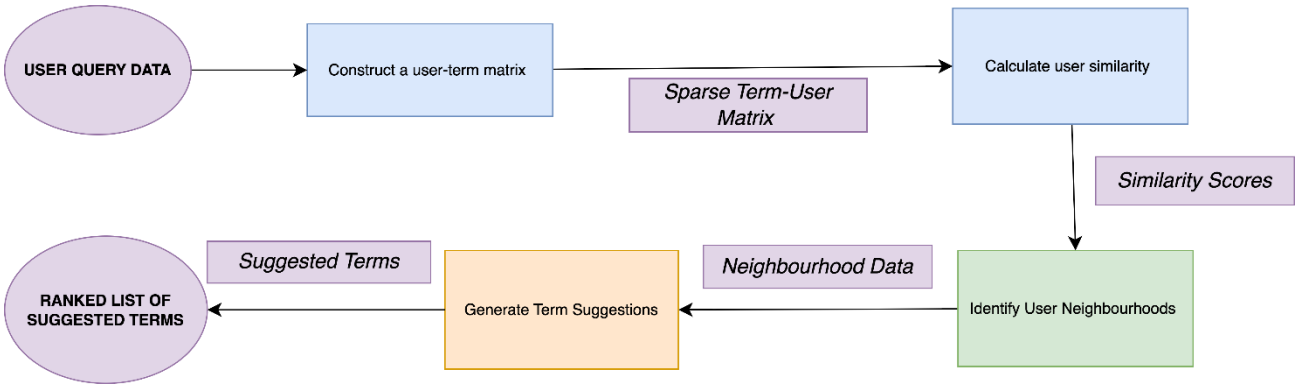

Figure 3: Flowchart of the proposed system

Advantages

- Recommendations are personalised and relevant
    - The reccomendations take into consideration the past behaviour of the target user, as well as the behaviour of similar users. This leads to most personalised and relevant suggestions.
- Discovery of related queries
    - Leveraging the similarities between users allows the system to suggest query terms to the target user that they may not have explicitly considered.
    - This provides the user with a diverse recommendation that might not otherwise be possible in systems that are narrower in scope.
- Dynamic system
    - The system is dynamic in that as the users of the system perform more searches, there is more data available to us in the user-term matrix, which facilitates a higher quality of reccomendations.

Disadvantages

- New users lack context (a 'cold start' problem) [5]
    - Suggestions for new users will be poor as we do not have a lot of data (query terms) for these target users. This makes it difficult to identify users who are similar accurately. They also might search for terms that haven't been searched for by other users in the past, which could lead to issues in accurately giving reccomendations for their search

- Sparsity issues
    - In a user-term matrix in a system such as a search engine, which would have a large number of users and a very diverse vocabulary of terms, the matrix is likely to be sparse. This can hinder us when calculating the similarity between users or trying to construct neighbourhoods of similar users.
- Computational cost
    - The cost of computing similarities between a high volume of users with a high volume of query terms in the matrix is computationally expensive; it has a computing financial cost and a longer computational time due to the vast size of the user-term matrix.

- Privacy concerns
    - Users might consider that their search queries are going to be used in recommending queries to other users. This might be relevant if they are searching for things that they consider to be personal information, and they do not want this search to be tied back to them without their consent,

References

1. Kopp, Olaf. “What Is BM25?” | *Kopp Consulting*, 25 Sept. 2024, www.kopp-online-marketing.com/what-is-bm25.
2. “Query Suggestions: Algolia.” *Algolia Documentation*, www.algolia.com/doc/guides/building-search-ui/ui-and-ux-patterns/query-suggestions/js/#optimize-query-suggestions. Accessed 25 Nov. 2024.
3. Otten, Neri Van. “Vector Space Model Made Simple with Examples & Tutorial in Python.” *Spot Intelligence*, 3 Oct. 2024, spotintelligence.com/2023/09/07/vector-space-model/.
4. **iDB. (2024). Understanding the Cosine Similarity Formula. [online] Available at: https://www.pingcap.com/article/understanding-the-cosine-similarity-formula/ [Accessed 25 Nov. 2024].**
5. **Wikipedia. (2020a). Cold start (recommender systems). [online] Available at: https://en.wikipedia.org/wiki/Cold_start_(recommender_systems).**